Practice 06: Vectorization

Petr Kuderov

Researcher at MIPT & AIRI

February 25, 2026

Changes in schedule and marks formulae

- 19 Марта 13-16ч: мини-коллоквиум по первому модулю
- Итоговый балл

$$\text{Итог} = \text{Округление}\left[\frac{\min(100, 0.1\ \text{А} + 0.2\ \text{К} + 0.4\ \text{ДЗ} + 0.4\ \text{Э})}{10}\right]$$

- Автоматы

$$\text{Накоп} = \frac{\min(60, 0.1\ \text{А} + 0.2\ \text{К} + 0.4\ \text{ДЗ})}{0.6} \geq 70$$

$$\Rightarrow \text{Итог} = \text{Округление}\left[\frac{\text{Накоп}}{10}\right]$$

n-step TD for off-policy

- requires correction (like Retrace or V-trace)

Experience collection vectorization

Vectorized gym[-nasium] API:

- TL;DR each element of the single environment API is vectorized individually (batched).
- agent plays in N parallel episodes
- each step an agent gets an array of observations, rewards and flags and selects an array of actions
- within `info` dict all keys contain vectorized values too
- dict/tuple observations keep structure, leaf elements are vectorized
- all parallel episodes are usually run in auto-reset mode
- there are several common options for auto-reset. **Know them! Know how to compute returns!**

Real vectorization:

- requires an explicit implementation
- a single env object
- all computations under the hood are vectorized
- jax implementations even allow running an env on GPU

Pseudo-vectorization (`SyncVectorEnv`, `AsyncVectorEnv`):

- just a useful wrapper over N independent envs. Still faster training due to batching on the agent side
- for lightweight, fast envs use Sync version, because Async has some overhead for multithreading.

RL agents with RNNs

What to remember

- LSTM/GRU have XxxCell versions in torch. They are better suitable for single layer, online usage in RL.
- GRU is usually is good enough and a bit faster than LSTM
- Correctly work with states: init, reset on episode end, detach after BPTT step
- Sometimes doing some warmup (w/o gradients enabled) is beneficial
- For replay buffer, you don’t store hiddens — you have to replay some rollouts
  - i.e. sample N T-step rollouts for the batch
  - init hiddens
  - replay them: T-step for loop with grads enabled
  - optionally, you sample T+K step rollouts and do K warmup steps w/o gradients

Practice

1. Based on DQN HW notebook
   - make Double DQN w/ Experience Replay Buffer
   - make flagged option to fill buffer up, make a single update, then flush buffer (to emulate on-policy)
2. Vectorize experience collection
   - learn how to work with vectorized envs
   - show plot, measure fps, compare w/ non-vectorized version w/ and w/o experience replay
3. Add RNN
   - continue vectorized experience collection
   - add RNN encoder to your agent
   - important part: state accumulation and reset
   - how to store data in replay buffer
   - how to make an update (add warmup)
   - to test:
     - cut out velocities the observation to make env POMDP
     - compare w/ non-RNN version